

SET-A

Name:	ID:	Section:
-------	-----	----------

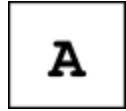
Question 1 [C01] [10 Points]

Write a Java program that takes an integer input N from the user and prints a pattern given below:

Input	Output
N = 5	<pre> 12345 12345 12345 12345 12345 123456789 1*****7 1***5 1*3 1 </pre>
N = 7	<pre> 1234567 1234567 1234567 1234567 1234567 1234567 1234567 1234567 12345678910111213 1*****11 1*****9 1*****7 1***5 1*3 1 </pre>



CSE110: Programming Language I (Lab)
Spring 2025
Lab Exam - 02
Duration: 25 Minutes



Name:	ID:	Section:
-------	-----	----------

SET-B

Name:	ID:	Section:
-------	-----	----------

Question 1 [C01] [10 Points]

Write a Java program that takes an integer input N from the user and prints a pattern given below:

Input	Output
N = 5	<pre> 1 1*3 1***5 1*****7 123456789 12345 12345 12345 12345 12345 </pre>
N = 7	<pre> 1 1*3 1***5 1*****7 1*****9 1*****11 12345678910111213 1234567 1234567 1234567 1234567 1234567 1234567 1234567 </pre>



CSE110: Programming Language I (Lab)
Spring 2025
Lab Exam - 02
Duration: 25 Minutes



Name:	ID:	Section:
-------	-----	----------

Solution & Rubric

In case of any circumstance not present in the rubrics, feel free to discuss in the assigned slack channel. This rubric is provided to maintain a similar marking scheme throughout all sections.

SET - [A]

SOLUTION

```
import java.util.Scanner;

public class NumberPattern {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);

        int n = sc.nextInt();
        for (int i = 1; i <= n; i++) {
            for (int j = 1; j <= n; j++) {
                System.out.print(j);
            }
            System.out.println();
        }
        for (int i = n; i >= 1; i--) {
            for (int space = 1 ; space <= n-i; space++){
                System.out.print(" ");
            }
            for (int j = 1 ; j <= 2 * i - 1 ; j++){
                if (i == n || i == 1 || j == 1 || j == 2 * i - 1){
                    System.out.print(j);
                }
                else{
                    System.out.print("*");
                }
            }
            System.out.println();
        }
    }
}
```

SET - [B]

SOLUTION


```
import java.util.Scanner;

public class NumberPatternB {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);

        int n = sc.nextInt();
        for (int i = 1; i <= n; i++) {
            for (int space = 1 ; space <= n-i; space++){
                System.out.print(" ");
            }
            for (int j = 1 ; j <= 2 * i - 1 ; j++){
                if (i == n || i == 1 || j == 1 || j == 2 * i - 1){
                    System.out.print(j);
                }
                else{
                    System.out.print("*");
                }
            }
            System.out.println();
        }
        for (int i = 1; i <= n; i++) {
            for (int j = 1; j <= n; j++) {
                System.out.print(j);
            }
            System.out.println();
        }
    }
}
```

SET-A

Name:	ID:	Section:
-------	-----	----------

Question 1 [C02] [10 Points]

A secret network system logs packet identifiers as a string of **lowercase letters**. During stable transmission, packet IDs appear in **continuous alphabetical order**, where each next character is exactly **one greater than the previous one**. Your task is to: Find the **longest continuous alphabetical sequence** in the string. If the **length of that sequence is greater than 5**, convert the sequence to **UPPERCASE** and print **every alternate character**, starting from the **first character**. If the length is **5 or less**, print the sequence normally (no skipping, no case change)

- You may only use `length()`, `charAt()`, loops, and conditions

Sample Input	Sample Output
xyzmnopqrstuvwxyz	MOQSUW
awdxyz	xyz

SET-B

Name:	ID:	Section:
-------	-----	----------

Question 1 [C02] [10 Points]

A DNA analysis system stores scan data as a string of **uppercase letters**. When the scanner works correctly, a **valid DNA signal** appears as a sequence of characters that follow **continuous alphabetical order**, meaning each next character is exactly one letter after the previous one. Find the **longest continuous alphabetical sequence** in the given string. If the **length of the sequence is smaller than 6**, convert the sequence to **lowercase** and print **every alternate character**, starting from the **first character** of the sequence. If the length is **6 or more**, print the sequence normally. (no skipping, no case change)

- You may only use `length()`, `charAt()`, loops, and conditions

Sample Input	Sample Output
XXMNOPQRSTUVWXYZABC	MNOPQRSTUVWXYZ
ABXYZ	xz

Solution & Rubric

In case of any circumstance not present in the rubrics, feel free to discuss in the assigned slack channel. This rubric is provided to maintain a similar marking scheme throughout all sections.

SET - [A]

SOLUTION

```
import java.util.Scanner;
class setA{
    public static void main(String[] args) {
        Scanner input=new Scanner(System.in);
        String st=input.nextLine();
        int currentStartIndex=0;
        int currentLength=1;
        int maxLength=0;
        int maxStartIndex=0;

        //identify the longest continuous alphabetical sequence
        for(int i =0; i<st.length()-1;i++){
            if(st.charAt(i)==(char) (st.charAt(i+1)-1)){
                currentLength++;
            }
            else {
                currentLength =1;
                currentStartIndex = i+1;
            }
            if(currentLength>maxLength ){
                maxLength=currentLength ;
                maxStartIndex=currentStartIndex ;
            }
        }

        //Correct conditional logic for output formatting (case change and skipping)
        int step=2;
        int caseShift=32;
        if(maxLength<=5){
            step=1; // no skipping
            caseShift=0; // no case change
        }
        for (int i = maxStartIndex ;i <maxStartIndex+maxLength; i=i+step) {
            char ch=st.charAt(i);
            ch=(char)(ch-caseShift);
            System.out.print(ch);

        }
    }
}
```

Marks

		[10]
		1
		1.5
		4
		3.5

SET - [B]

SOLUTION

```
import java.util.Scanner;

class setB {
    public static void main(String[] args) {
        Scanner input = new Scanner(System.in);
        String st = input.nextLine();

        int currentStartIndex = 0;
        int currentLength = 1;
        int maxLength = 0;
        int maxStartIndex = 0;

        //identify the longest continuous alphabetical sequence
        for (int i = 0; i < st.length() - 1; i++) {
            if (st.charAt(i) == st.charAt(i + 1) - 1) {
                currentLength++;
            } else {
                currentLength = 1;
                currentStartIndex = i + 1;
            }

            if (currentLength > maxLength) {
                maxLength = currentLength;
                maxStartIndex = currentStartIndex;
            }
        }

        //Correct conditional logic for output formatting (case change and skipping)
        int step;
        int caseShift;
        if (maxLength < 6) {
            step = 2;
            caseShift = 32;
        } else {
            step = 1;    // no skipping
        }
    }
}
```



```
    caseShift = 0; // no case change
}

for (int i = maxStartIndex; i < maxStartIndex + maxLength; i += step) {
    char ch = st.charAt(i);
    ch = (char)(ch + caseShift);
    System.out.print(ch);
}
}
```

SET-A

Name:	ID:	Section:
-------	-----	----------

Question 1 [C01] [10 Points]

In a live cricket match analytics system, individual player scores are being recorded in real time as the match progresses. The system can track a maximum of 11 players.

Each time a player's score is entered, the system must immediately display the current Top-3 highest scores, showing the three highest run-scorers in descending order. If fewer than 3 player scores have been entered, the system should display:

Minimum 3 scores needed to see the leaderboard assume that the user will give a minimum 4 scores.

Score input continues until:

- The user enters -1, or
- A total of 11 player scores have been recorded

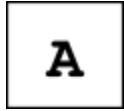
(whichever occurs first)

After input ends, the system must display the final Top-3 highest scores (Top Run Scorers).

Sample Input	Sample Output
45 78 30 92 65 21 -1	Minimum 3 scores needed to see the leaderboard Minimum 3 scores needed to see the leaderboard Current Top 3: 78 45 30 Current Top 3: 92 78 45 Current Top 3: 92 78 65 Current Top 3: 92 78 65 Final Top 3: 92 78 65
50 88 34 75 60 41 95 -1	Minimum 3 scores needed to see the leaderboard Minimum 3 scores needed to see the leaderboard Current Top 3: 88 50 34 Current Top 3: 88 75 50 Current Top 3: 88 75 60 Current Top 3: 88 75 60 Current Top 3: 95 88 75 Final Top 3: 95 88 75



CSE110: Programming Language I (Lab)
Spring 2026
Lab Exam - 4
Duration: 25 Minutes



Name:	ID:	Section:
-------	-----	----------

SET-B

Name:	ID:	Section:
-------	-----	----------

Question 1 [C01] [10 Points]

In a data analysis system, a sequence of integer values is stored in an array. The system is required to analyze short-term trends using three consecutive elements at a time. The program first takes an integer N as input from the user, which represents the size of the array. Then, the user enters N integer values to fill the array.

After calculating each average, the system must immediately display the current maximum average of all consecutive triplets seen so far.

If fewer than 3 elements have been entered, the system should display: "Minimum 3 numbers needed to calculate average"

Assume that the user will give a minimum of 4 numbers. Processing continues until all inputs are entered. After input ends, the system must display the final maximum average of all consecutive triplets.

Sample Input	Sample Output
5 10 20 5 8 15	Minimum 3 numbers needed to calculate average Minimum 3 numbers needed to calculate average Current Max Average: 11.67 Current Max Average: 11.67 Final Max Average: 11.67
6 3 7 2 9 4 10	Minimum 3 numbers needed to calculate average Minimum 3 numbers needed to calculate average Current Max Average: 4.0 Current Max Average: 6.0 Current Max Average: 6.67 Current Max Average: 7.67 Final Max Average: 7.67



CSE110: Programming Language I (Lab)
Spring 2025
Lab Exam - 04
Duration: 25 Minutes

B

Name:	ID:	Section:
-------	-----	----------

Solution & Rubric

In case of any circumstance not present in the rubrics, feel free to discuss in the assigned slack channel. This rubric is provided to maintain a similar marking scheme throughout all sections.

SET - [A]

SOLUTION

```
import java.util.Scanner;

public class SetASolve {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);

        int[] scores = new int[11];
        int count = 0;
        int top1 = 0, top2 = 0, top3 = 0;

        while (count < 11) {
            int score = sc.nextInt();

            if (score == -1) {
                break;
            }

            scores[count] = score;
            count++;

            if (count < 3) {
                System.out.println("Minimum 3 scores needed to see the leaderboard");
            }
            else {
                for (int i = 0; i < count; i++) {
                    int val = scores[i];

                    if (val > top1) {
                        top3 = top2;
                        top2 = top1;
                        top1 = val;
                    }
                    else if (val > top2) {
                        top3 = top2;
                        top2 = val;
                    }
                    else if (val > top3) {
                        top3 = val;
                    }
                }

                System.out.println("Current Top 3: " + top1 + " " + top2 + " " + top3);
            }
        }
        System.out.println("Final Top 3: " + top1 + " " + top2 + " " + top3);
    }
}
```

SET - [B]**SOLUTION**

```
import java.util.*;

public class SetBSolve {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);

        int n = sc.nextInt();
        int[] arr = new int[n];

        for (int i = 0; i < n; i++) {
            arr[i] = sc.nextInt();
        }

        double maxAvg = -0.0;

        for (int i = 0; i < n; i++) {
            if (i < 2) {
                System.out.println("Minimum 3 numbers needed to calculate average");
            }
            else {
                int sum = arr[i] + arr[i - 1] + arr[i - 2];
                double avg = sum / 3.0;

                if (avg > maxAvg) {
                    maxAvg = avg;
                }

                System.out.println("Current Max Average: " + maxAvg);
            }
        }

        System.out.println("Final Max Average: " + maxAvg);
    }
}
```